CSE-3079

EMBEDDED IoT SYSTEMS

WEEK 04

Hardware Timers & Timer-Based Button Debouncing

FROM WEEK 3 → WEEK 4

Week 3 established GPIO interrupts, ISR execution rules and mechanical switch bounce. Week 4 adds hardware timers so the ESP32 can measure a precise interval and validate a button without blocking the main application.

Course	Class	Semester	Instructor
CSE-3079 Embedded IoT Systems	CS 5th A, B	Fall 2026	Dr. Nasir Mahmood

Student Guide • Lecture + Practical Laboratory

Contents

1. Week 4 Position in the Course
2. Learning Outcomes
3. Week 3 Recap: Why Timers Are Needed
4. Hardware Timer Fundamentals
5. Timer Frequency, Tick, Counter and Alarm
6. Periodic vs One-Shot Timers
7. ESP32 Timer API — Current Arduino-ESP32 3.x
8. Practical 1 — Periodic Timer LED
9. Mechanical Switch Bounce
10. Debouncing Strategies
11. Timer-Based Non-Blocking Debounce
12. GPIO ISR + Timer ISR Architecture
13. Complete ESP32 Timer-Debounce Implementation
14. Line-by-Line Code Explanation
15. Timing Calculations and Design Exercises
16. Practical Laboratory Tasks
17. Common Errors and Debugging
18. Technical Short Questions & Answers
19. MCQs
20. Viva Questions
21. Mini Assessment / Exit Ticket
22. Week 4 Takeaway
23. References

Timer operation	Arduino-ESP32 2.x — Old API	Arduino-ESP32 3.x — Current API
Create timer	timerBegin(0, 80, true)	timerBegin(1000000)
Attach ISR	timerAttachInterrupt(timer, &onTimer, true)	timerAttachInterrupt(timer, &onTimer)
Configure alarm	timerAlarmWrite(timer, 1000000, true)	timerAlarm(timer, 1000000, true, 0)
Enable alarm	timerAlarmEnable(timer)	Included in timerAlarm()
Disable alarm	timerAlarmDisable(timer)	Use current timer control API as required
Set counter	timerWrite(timer, value)	timerWrite(timer, value)
Read counter	timerRead(timer)	timerRead(timer)
Frequency setup	Programmer specifies timer number/divider/count direction	Programmer requests desired frequency directly

```
// Arduino-ESP32 2.x
timerAlarmWrite(timer, 1000000, true);
timerAlarmEnable(timer);

// Arduino-ESP32 3.x
timerAlarm(timer, 1000000, true, 0);
```

```
// OLD — Arduino-ESP32 2.x
timer = timerBegin(0, 80, true);
timerAttachInterrupt(timer, &onTimer, true);
timerAlarmWrite(timer, 1000000, true);
timerAlarmEnable(timer);
```

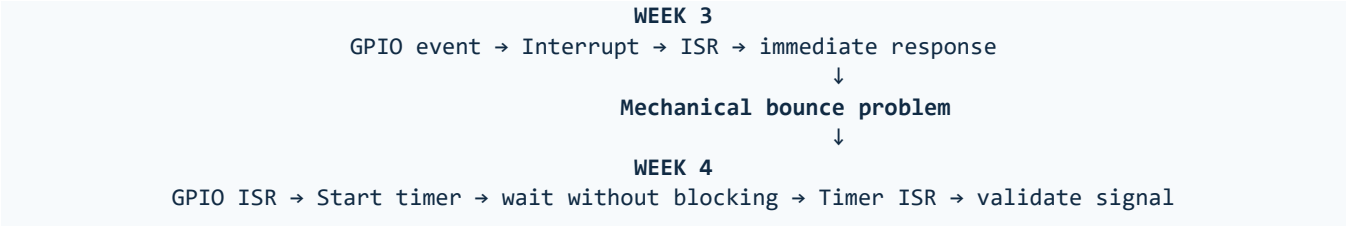
```
// NEW — Arduino-ESP32 3.x
timer = timerBegin(1000000);
timerAttachInterrupt(timer, &onTimer);
timerAlarm(timer, 1000000, true, 0);

// OLD — Arduino-ESP32 2.x
timerWrite(debounceTimer, 0);
timerAlarmWrite(debounceTimer, 50000, false);
timerAlarmEnable(debounceTimer);

// NEW — Arduino-ESP32 3.x
timerWrite(debounceTimer, 0);
timerAlarm(debounceTimer, 50000, false, 0);
```

1. Week 4 Position in the Course

The central idea of Week 4 is to combine two event mechanisms introduced in Week 3: a GPIO interrupt detects a physical transition, while a hardware timer provides a controlled time interval before the signal is validated.



CORE IDEA

A GPIO interrupt tells the ESP32 that something happened. A timer tells the ESP32 when enough time has passed.

2. Learning Outcomes

- Explain the purpose and operation of a hardware timer.
- Differentiate a hardware timer from delay()-based waiting.
- Define timer frequency, tick, counter, alarm and auto-reload.
- Calculate timer intervals from frequency and alarm ticks.
- Configure an ESP32 hardware timer using the current Arduino-ESP32 timer API.
- Generate periodic and one-shot timer events.
- Explain why a mechanical button can create multiple electrical transitions.
- Explain why delay() is inappropriate inside an ISR.
- Implement timer-based, non-blocking button debouncing.
- Combine a GPIO ISR with a timer ISR.
- Use volatile state variables correctly when data is shared with an ISR.
- Debug timer and debounce programs in PlatformIO.

3. Week 3 Recap: Why Timers Are Needed

Week 3 introduced GPIO interrupts and the rules for writing short interrupt service routines. The Week 3 slides also show the physical bounce produced by mechanical contacts and the need for debouncing. The bounce interval is presented as roughly 5–50 ms in the course material.

THE PROBLEM

The ESP32 sees electrical edges, not human intention. One physical press can therefore produce several interrupt events.

```
One human press
        ↓
Mechanical contacts vibrate
        ↓
HIGH ⇔ LOW ⇔ HIGH ⇔ LOW ...
        ↓
Several GPIO interrupt edges
        ↓
One intended action may be triggered multiple times
```

The Week 3 material compares hardware filtering, software delay and timer-based debounce, with timer-based debounce presented as the non-blocking approach developed in the course.

- GPIO interrupt: event detection.
- ISR: immediate, short response.
- Timer: precise elapsed-time mechanism.
- Debounce: reject unstable transitions before accepting a button press.

Week 3 established the interrupt-driven programming model used by the rest of this course. A GPIO interrupt allows the ESP32 to respond to a pin transition immediately instead of continuously polling the pin in loop(). Week 4 builds on this foundation by adding a hardware timer to measure the debounce interval.

3.1 What Is an Interrupt?

An interrupt is a hardware or software event that temporarily transfers program execution to an Interrupt Service Routine (ISR). After the ISR finishes, normal program execution can continue.

3.2 GPIO Interrupt Sequence

For a button connected to a GPIO, the basic sequence is:

- Button state changes.
- The GPIO hardware detects the configured edge.
- The ESP32 enters the registered ISR.
- The ISR performs a short, immediate response or updates a shared state/flag.
- The ISR returns and the main program continues.

3.3 Basic ESP32 GPIO Interrupt Example

Example: a push button on GPIO 4 toggles an LED on GPIO 2. The button uses INPUT_PULLUP, so a press connected to GND produces a FALLING edge.

Example 1 — GPIO Interrupt with Button and LED

```
#include <Arduino.h>

#define BUTTON_PIN 4
#define LED_PIN 2

volatile bool ledState = false;

void ARDUINO_ISR_ATTR onButtonISR()
{
    ledState = !ledState;
```

```
}

void setup()
{
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(LED_PIN, OUTPUT);

    attachInterrupt(
        digitalPinToInterrupt(BUTTON_PIN),
        onButtonISR,
        FALLING
    );
}

void loop()
{
    digitalWrite(LED_PIN, ledState);
}
```

3.4 Understanding attachInterrupt()

Code element	Meaning
digitalPinToInterrupt(BUTTON_PIN)	Converts the GPIO number to the interrupt source used by attachInterrupt().
onButtonISR	The function executed when the configured interrupt occurs.
FALLING	Trigger when the signal changes from HIGH to LOW.
INPUT_PULLUP	Uses the internal pull-up; released = HIGH and pressed to GND = LOW.

3.5 Interrupt Trigger Modes

Mode	Meaning
RISING	Trigger when signal changes LOW → HIGH.
FALLING	Trigger when signal changes HIGH → LOW.
CHANGE	Trigger on either LOW → HIGH or HIGH → LOW.
HIGH	Trigger while the pin remains HIGH; use only when appropriate for the hardware/application.

3.6 ISR Rules from Week 3

- Keep the ISR short and deterministic.
- Do not use delay() inside an ISR.
- Avoid long Serial.print() operations inside an ISR.
- Avoid lengthy calculations and blocking operations.
- Use volatile for variables shared between ISR and normal program flow when appropriate.
- Set a flag or update a small state variable in the ISR, then perform larger application work in loop().

3.7 Better Architecture: ISR Sets a Flag

For larger applications, the ISR should usually signal an event rather than perform the complete application action. This architecture is especially important for the Week 4 timer-based debounce design.

Example 2 — ISR Sets a Flag; loop() Performs the Work

```
#include <Arduino.h>
```

```
#define BUTTON_PIN 4
#define LED_PIN 2

volatile bool buttonEvent = false;

void ARDUINO_ISR_ATTR onButtonISR()
{
    buttonEvent = true;
}

void setup()
{
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(LED_PIN, OUTPUT);

    attachInterrupt(
        digitalPinToInterrupt(BUTTON_PIN),
        onButtonISR,
        FALLING
    );
}

void loop()
{
    if (buttonEvent)
    {
        buttonEvent = false;
        digitalWrite(LED_PIN, !digitalRead(LED_PIN));
    }

    // Other application work can continue here.
}
```

3.8 The Week 3 Button Problem: Mechanical Bounce

A mechanical button does not always produce one clean electrical transition. The contacts can vibrate for a short period, creating several HIGH/LOW transitions. Therefore, one physical press may generate several GPIO interrupt events.

This is the reason Week 4 introduces a timer: the GPIO ISR detects the candidate edge, the hardware timer provides a controlled debounce interval, and a timer ISR validates the button after the interval.

3.9 Week 3 → Week 4 Connection

Architecture Transition

```
WEEK 3
Button edge
   ↓
GPIO interrupt
   ↓
Short GPIO ISR
   ↓
Respond / set flag

WEEK 4
Button edge
   ↓
GPIO interrupt
   ↓
Short GPIO ISR
   ↓
```

```
Start one-shot hardware timer
   ↓
Main loop continues
   ↓
Timer alarm
   ↓
Timer ISR
   ↓
Validate button
   ↓
Accept / reject press
```

Core idea: an interrupt detects that an event happened; a timer provides a precise elapsed-time interval. Week 4 combines both mechanisms to create non-blocking button debouncing.

4. Hardware Timer Fundamentals

A hardware timer is a dedicated peripheral that counts clock ticks independently of the normal application flow. When the counter reaches a configured alarm value, the timer can generate an event or interrupt.

```
Clock source / timer clock
            ↓
Timer counter: 0, 1, 2, 3, ...
            ↓
Alarm threshold reached
            ↓
Timer event / interrupt
            ↓
        Timer ISR
```

Why not simply use delay()?

Blocking delay()	Hardware timer
Stops the current flow while waiting.	Counts time in a dedicated timer peripheral.
Poor fit inside an ISR.	Can generate an interrupt when the interval expires.
Can make the application less responsive.	Allows the main loop to continue other work.
Simple for very basic sketches.	Better for event-driven timing and precise intervals.

EMBEDDED-SYSTEM PRINCIPLE
Do not confuse 'waiting for time to pass' with 'making the CPU do nothing'. A timer allows time to pass while the application continues executing.

5. Timer Frequency, Tick, Counter and Alarm

5.1 Timer frequency

Frequency is the number of timer ticks generated per second.

```
1 MHz = 1,000,000 Hz
1 tick = 1 / 1,000,000 second
       = 1 microsecond (µs)
```

5.2 Timer period

The period of one timer tick is:

```
T_tick = 1 / f
```

For a timer configured at 1 MHz:

```
T_tick = 1 / 1,000,000
       = 1 µs
```

5.3 Alarm calculation

If N is the alarm count and f is the timer frequency:

```
T_event = N / f

Therefore:
N = T_event × f
```

Required interval	Equivalent time	Alarm ticks @ 1 MHz
100 µs	100 µs	100
1 ms	1,000 µs	1,000
10 ms	10,000 µs	10,000
50 ms	50,000 µs	50,000
100 ms	100,000 µs	100,000
500 ms	500,000 µs	500,000
1 s	1,000,000 µs	1,000,000
2 s	2,000,000 µs	2,000,000

CLASSROOM CHECK

If the timer resolution is 1 µs per tick, a 50 ms debounce interval requires 50,000 ticks.

6. Periodic vs One-Shot Timers

6.1 Periodic / auto-reload timer

After the alarm event, the timer automatically reloads and continues. This is appropriate for repeated actions.



Examples:

- Blink an LED every 1 second.
- Sample a sensor every 100 ms.
- Update a display periodically.

6.2 One-shot timer

A one-shot timer generates one alarm event and does not automatically repeat. This is ideal when a single validation event is required after a delay.

```
Button edge
    ↓
Start one-shot timer
    ↓
   50 ms
    ↓
 Timer ISR
    ↓
Validate button
    ↓
   Stop
```

Examples:

- Button debounce validation.
- Delayed device action.
- Timeout after a single event.

DESIGN RULE
Periodic timer = repeated event. One-shot timer = single delayed event. Timer-based debounce normally uses one-shot behavior.

7. ESP32 Timer API — Current Arduino-ESP32 3.x

For the current Arduino-ESP32 API, timerBegin() takes the requested timer frequency, timerAttachInterrupt() connects a callback, and timerAlarm() configures the alarm and enables it.

Function	Purpose	Typical use in Week 4
timerBegin(frequency)	Create/configure timer at requested frequency.	timerBegin(1000000)
timerAttachInterrupt(timer, callback)	Attach timer ISR callback.	timerAttachInterrupt(timer, &onTimer)
timerAlarm(timer, value, autoreload, reload_count)	Configure alarm and enable timer alarm.	50,000 ticks, one-shot
timerWrite(timer, value)	Set timer counter value.	Reset counter to zero
timerRead(timer)	Read current counter value.	Optional debugging
timerEnd(timer)	End/free timer.	Cleanup when timer is no longer required

API VERSION NOTE
Older ESP32 examples may use timerBegin() with multiple parameters and timerAlarmWrite()/timerAlarmEnable(). Arduino-ESP32 3.x simplified the timer API. Use the current API consistently in this course.

Minimal current-API timer structure

```
hw_timer_t *timer = NULL;

void ARDUINO_ISR_ATTR onTimer()
{
    // Keep this ISR short.
}

void setup()
{
    timer = timerBegin(1000000);             // 1 MHz
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 1000000, true, 0);     // 1 s, periodic
}
```

8. Practical 1 — Periodic Timer LED

Goal: toggle an LED every one second using a hardware timer.

```
#include <Arduino.h>

#define LED_PIN 2

hw_timer_t *timer = NULL;
volatile bool ledState = false;

void ARDUINO_ISR_ATTR onTimer()
{
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
}

void setup()
{
    pinMode(LED_PIN, OUTPUT);

    timer = timerBegin(1000000);        // 1 MHz = 1 µs/tick
    timerAttachInterrupt(timer, &onTimer);

    timerAlarm(timer, 1000000, true, 0);
}

void loop()
{
    // Main program remains available for other work.
}
```

Expected operation

Timer starts
↓
1,000,000 ticks
↓
Timer ISR
↓
LED toggles
↓
Auto-reload
↓

```
Repeat every 1 s
```

Student modifications

1. Change the period from 1 s to 500 ms.
2. Change it to 250 ms.
3. Change it to 2 s.
4. Add a second task in loop() and observe that the timer continues to operate.

9. Mechanical Switch Bounce

A mechanical push button is not an ideal digital device. Its contacts can physically vibrate when opening or closing. During this short transition, the electrical signal may alternate between logic levels.

```
Stable HIGH
      ↓
Press button
      ↓
Contact vibration
      ↓
HIGH → LOW → HIGH → LOW → HIGH → LOW ...
      ↓
Stable LOW
```

The course Week 3 slides depict a bounce interval of approximately 5–50 ms. The exact duration depends on the switch, hardware and physical conditions.

IMPORTANT
Debouncing is not about making the button electrically perfect. It is about deciding which part of the observed signal represents the user's intended action.

10. Debouncing Strategies

Method	Basic idea	Advantages	Limitations
Hardware RC filter	Smooth the electrical transition with resistor/capacitor circuitry.	Can suppress physical noise before software sees it.	Requires additional components; threshold behavior must be considered.
Software delay	Wait a short time after detecting an edge.	Very simple to understand.	Blocking; unsuitable inside an ISR.
Timer-based debounce	Start a one-shot timer and validate after the interval.	Non-blocking; event-driven; good fit with interrupts.	Requires timer configuration and careful ISR/state design.

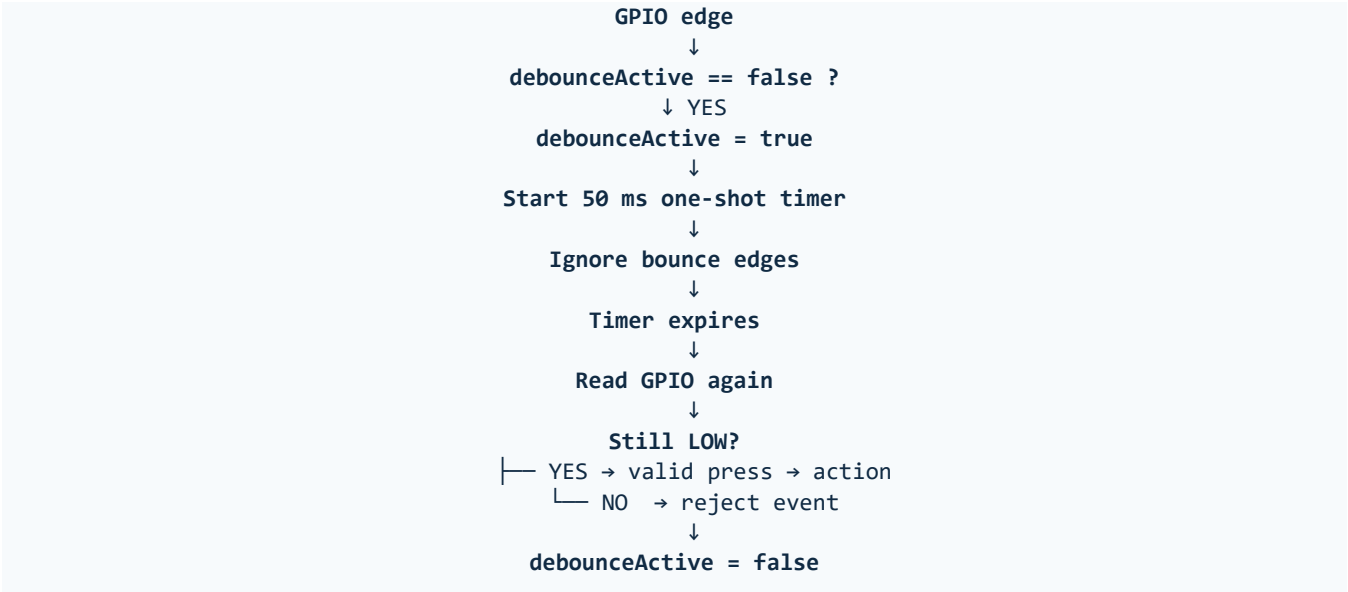
Why timer-based debounce fits the course architecture

The Week 3 slide sequence identifies timer-based debounce as the non-blocking path. The GPIO ISR reacts immediately, while the hardware timer provides the debounce interval without pausing the main application.

11. Timer-Based Non-Blocking Debounce

The algorithm is:

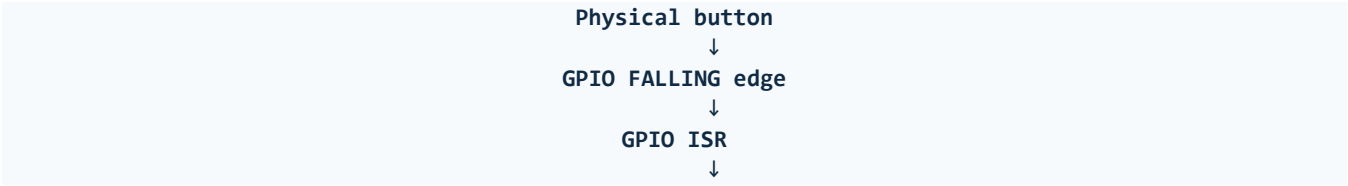
1. Configure the button with INPUT_PULLUP.
2. Attach a GPIO interrupt on the falling edge.
3. When the first edge occurs, check debounceActive.
4. If debounceActive is false, set it to true.
5. Start a 50 ms one-shot hardware timer.
6. Ignore additional GPIO edges while debounceActive is true.
7. When the timer expires, execute the timer ISR.
8. Read the button again.
9. If the button is still LOW, accept it as a valid press.
10. Perform the required action.
11. Clear debounceActive so the next press can be accepted.



12. GPIO ISR + Timer ISR Architecture

Two different interrupt sources are involved.

Interrupt source	Cause	Role
GPIO interrupt	Physical button transition.	Detect candidate press and start debounce interval.
Timer interrupt	Timer reaches alarm value.	Validate the button after the debounce interval.



```
Start 50 ms one-shot timer
        ↓
Main loop continues doing other work
        ↓
    Timer alarm
        ↓
    Timer ISR
        ↓
   Read button
        ↓
  Accept / reject
```

KEY DISTINCTION

GPIO interrupt answers: 'Did an edge occur?' Timer interrupt answers: 'Has the debounce interval elapsed?'

13. Complete ESP32 Timer-Debounce Implementation

```
#include <Arduino.h>

#define BUTTON_PIN 4
#define LED_PIN    2

hw_timer_t *debounceTimer = NULL;

volatile bool debounceActive = false;

void ARDUINO_ISR_ATTR onButtonISR()
{
    if (!debounceActive)
    {
        debounceActive = true;

        // 50 ms one-shot timer
        timerWrite(debounceTimer, 0);
        timerAlarm(debounceTimer, 50000, false, 0);
    }
}

void ARDUINO_ISR_ATTR onDebounceTimer()
{
    if (digitalRead(BUTTON_PIN) == LOW)
    {
        // Valid press
        digitalWrite(LED_PIN, !digitalRead(LED_PIN));
    }

    debounceActive = false;
}

void setup()
{
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(LED_PIN, OUTPUT);

    // 1 MHz → 1 µs per tick
    debounceTimer = timerBegin(1000000);
```

```
    timerAttachInterrupt(
        debounceTimer,
        &onDebounceTimer
    );

    attachInterrupt(
        digitalPinToInterrupt(BUTTON_PIN),
        onButtonISR,
        FALLING
    );
}

void loop()
{
    // Main application remains non-blocking.
}
```

HARDWARE ASSUMPTION

This example assumes a push button connected between GPIO 4 and GND, using the ESP32's internal pull-up. When the button is released the input is HIGH; when pressed it is LOW.

14. Line-by-Line Code Explanation

Code element	Meaning
hw_timer_t *debounceTimer = NULL;	Declares a handle/pointer for the hardware timer.
volatile bool debounceActive = false;	Stores the debounce state. volatile is appropriate because the value is accessed in interrupt context.
onButtonISR()	Runs when the button produces the configured falling edge.
if (!debounceActive)	Prevents multiple bounce edges from starting multiple debounce sequences.
debounceActive = true;	Locks out additional candidate edges during the debounce interval.
timerWrite(debounceTimer, 0);	Resets the timer counter to zero before the debounce interval.
timerAlarm(..., 50000, false, 0);	Requests a 50,000-tick one-shot alarm. At 1 MHz this is 50 ms.
onDebounceTimer()	Runs when the 50 ms timer alarm expires.
digitalRead(BUTTON_PIN) == LOW	Checks whether the button is still physically asserted after the interval.
debounceActive = false;	Re-arms the system for the next button press.

ISR discipline

- Keep both ISRs short.
- Avoid delay() inside an ISR.
- Avoid lengthy Serial operations inside an ISR.
- Use shared state carefully.
- Move larger application work to loop() or another task when possible.

15. Timing Calculations and Design Exercises

Exercise A — Calculate alarm ticks

1. At 1 MHz, calculate the alarm value for 25 ms.
2. At 1 MHz, calculate the alarm value for 75 ms.
3. At 1 MHz, calculate the alarm value for 250 ms.
4. At 2 MHz, calculate the alarm value for 50 ms.

Exercise B — Reverse calculation

1. A 1 MHz timer has an alarm value of 20,000. What is the interval?
2. A 1 MHz timer has an alarm value of 250,000. What is the interval?
3. A 500 kHz timer has an alarm value of 25,000. What is the interval?

Exercise C — Design choice

1. Would you use a periodic or one-shot timer to blink an LED every second? Why?
2. Would you use a periodic or one-shot timer to validate a button 50 ms after an edge? Why?
3. Why is a timer better than delay(50) inside a GPIO ISR?

ANSWER CHECK

At 1 MHz, every tick is 1 µs. Therefore 25 ms = 25,000 ticks, 75 ms = 75,000 ticks, and 250 ms = 250,000 ticks.

16. Practical Laboratory Tasks

Lab 1 — Periodic Hardware Timer

Item	Requirement
Objective	Blink/toggle an LED using a periodic hardware timer.
Timer	1 MHz
Period	1 second
Expected result	LED changes state every 1 second.
Extension	Add a second timer-controlled behavior or change the period.

Lab 2 — One-Shot Timer

Item	Requirement
Objective	Use a button to start a one-shot timer.
Input	Button on GPIO 4 using INPUT_PULLUP.
Timer	1 MHz
Delay	2 seconds
Expected result	Timer ISR performs an action once after 2 seconds.

Lab 3 — Timer-Based Button Debounce

Item	Requirement
Objective	Implement non-blocking debounce using GPIO ISR + one-shot timer.
Debounce interval	50 ms
GPIO event	FALLING
Validation	Read the button again after 50 ms.
Expected result	One valid action per physical press.

1. Lab 4 — Compare Before and After Debouncing

1. Create a GPIO interrupt program without debounce.
2. Press the button slowly several times and record the number of detected events.
3. Add the timer-based debounce logic.
4. Repeat the test.
5. Explain why the count changes.

Suggested submission evidence

- PlatformIO project folder.
- main.cpp source file.
- Short circuit/wiring diagram or photograph.
- Serial Monitor output where applicable.
- A short explanation of the timing calculation.
- One paragraph explaining why the debounce solution is non-blocking.

17. Common Errors and Debugging

Symptom	Likely cause	Check / fix
timerBegin() compilation error	Using an older API example with current Arduino-ESP32.	Use timerBegin(frequency) with the current core.
timerAlarmWrite() not found	Old timer API.	Use timerAlarm(timer, value, autoreload, reload_count).
Timer repeats unexpectedly	autoreload set true.	Use false for a one-shot debounce timer.
Button triggers multiple times	No debounce or debounce guard missing.	Use debounceActive + one-shot validation.
Button logic inverted	INPUT_PULLUP misunderstood.	Released = HIGH; pressed = LOW.
System behaves strangely after ISR	ISR doing too much work.	Keep ISR short; move application work out of ISR.
LED never changes	Wrong GPIO, wiring, or active-low hardware.	Check pin, ground, resistor/LED polarity and board pinout.
Timer interval incorrect	Wrong frequency/tick calculation.	Recalculate N = T × f and verify timerGetFrequency if needed.

PLATFORMIO CHECK
If a timer example behaves differently from the slides, first inspect the Arduino-ESP32 platform/core version in the build output.

The timer API changed significantly from 2.x to 3.x.

18. Technical Short Questions & Answers

1. What is a hardware timer?
A hardware peripheral that counts timer ticks and can generate an event when a configured alarm value is reached.

2. What is a timer tick?
One increment of the timer counter. Its duration depends on the configured timer frequency.

3. What is the duration of one tick at 1 MHz?
1 microsecond.

4. What is an alarm?
A configured counter value at which the timer generates its event/interrupt.

5. What is a periodic timer?
A timer that automatically reloads after an alarm and generates repeated events.

6. What is a one-shot timer?
A timer that generates one event after its interval and does not automatically repeat.

7. Why is a one-shot timer useful for debounce?
A button needs one validation event after the debounce interval, not a continuous stream of timer events.

8. What is mechanical bounce?
Rapid unwanted electrical transitions caused by physical vibration of mechanical switch contacts.

9. Why can a GPIO interrupt alone misinterpret a button press?
Because each bounce edge can satisfy the interrupt condition.

10. Why should delay() not be used inside an ISR?
It blocks execution while the ISR is active and violates the requirement that an ISR remain short and responsive.

11. What does debounceActive do?
It prevents additional bounce edges from starting new debounce sequences while one sequence is already active.

12. Why is volatile used for ISR-shared variables?
It tells the compiler that the value may change asynchronously and should not be treated as an unchanged cached value.

13. What is the debounce interval in the example?
50 ms.

14. At 1 MHz, how many ticks represent 50 ms?
50,000 ticks.

15. What is the role of the GPIO ISR in timer-based debounce?
Detect the candidate edge, activate the debounce state and start the one-shot timer.

16. What is the role of the timer ISR?
Validate the button state after the debounce interval.

17. What does INPUT_PULLUP mean?
The GPIO uses an internal pull-up so the normal/released state is HIGH; a button connected to GND produces LOW when pressed.

18. What is the difference between GPIO interrupt and timer interrupt?
GPIO interrupt is caused by a pin event; timer interrupt is caused by elapsed timer count reaching its alarm.

19. MCQs

1. At 1 MHz, one timer tick is:

A. 1 s

B. 1 ms

C. 1 µs

D. 1 ns

Answer: C

2. Which timer mode is normally used for a single debounce validation?

A. Periodic

B. One-shot

C. PWM

D. Continuous GPIO

Answer: B

3. A 50 ms interval at 1 MHz requires:

A. 500 ticks

B. 5,000 ticks

C. 50,000 ticks

D. 500,000 ticks

Answer: C

4. Which event normally starts the debounce sequence?

A. Timer alarm

B. GPIO edge

C. Wi-Fi packet

D. ADC conversion

Answer: B

5. What does timerAlarm(..., false, 0) request for the auto-reload field?

A. Repeat forever

B. One-shot behavior

C. Disable the GPIO

D. Reverse the counter

Answer: B

6. Which variable should be protected from compiler assumptions when changed asynchronously by an ISR?

A. const

B. static only

C. volatile

D. register

Answer: C

7. With INPUT_PULLUP and a button connected to GND, a pressed button reads:

A. HIGH

B. LOW

C. Floating

D. Analog 3.3 V

Answer: B

8. Which operation is least appropriate inside a short ISR?

A. Set a flag

B. Update a small state variable

C. Start/arm a timer

D. Perform long blocking work

Answer: D

20. Viva Questions

1. Draw the architecture of a hardware timer.
2. Explain the relationship between frequency and tick duration.
3. Why does a 1 MHz timer provide 1 µs resolution?
4. Differentiate periodic and one-shot timer operation.
5. Explain what happens when timerAlarm() is called.
6. Why is timer-based debounce non-blocking?
7. Why does a mechanical switch bounce?
8. Why can FALLING-edge interrupts produce multiple events during bounce?
9. Explain the purpose of debounceActive.
10. Why is the button configured as INPUT_PULLUP in the example?
11. Why is the button considered LOW when pressed?
12. Explain the difference between the GPIO ISR and timer ISR.
13. What would happen if autoreload were accidentally set to true for the debounce timer?
14. Why should Serial.print() and delay() generally be avoided inside an ISR?
15. How would you change the debounce interval from 50 ms to 20 ms?
16. How would you verify that the timer is actually configured at 1 MHz?
17. What is the effect of using an excessively long debounce interval?
18. How would you redesign the program if the button action were computationally expensive?

21. Mini Assessment / Exit Ticket

Students should answer these without opening the code:

1. In one sentence, define a hardware timer.
2. Calculate the alarm ticks for 30 ms at 1 MHz.
3. State one difference between periodic and one-shot timers.
4. Why is a GPIO interrupt alone unreliable for a mechanical button?
5. What does debounceActive protect against?
6. Why is delay() unsuitable inside an ISR?
7. What does the timer ISR check after 50 ms?
8. Draw the GPIO ISR → timer → timer ISR sequence.
9. State one reason for using volatile.
10. Give one real IoT application where timer-based event handling is useful.

22. Week 4 Takeaway

THE MASTER CONCEPT

Interrupts detect events; timers measure elapsed time; combining them creates precise, non-blocking event handling.

```
Physical event
      ↓
GPIO interrupt
      ↓
  Short ISR
      ↓
Start hardware timer
      ↓
Main program keeps running
      ↓
  Timer alarm
      ↓
  Timer ISR
      ↓
Validate / act
```

- Use GPIO interrupts for fast event detection.
- Use hardware timers for controlled time intervals.
- Use one-shot timers for single delayed validation.
- Use periodic timers for repeated events.
- Keep ISRs short and avoid blocking operations.
- Use a state/guard variable to prevent repeated debounce sequences.
- For a button with INPUT_PULLUP, a press to GND normally produces a LOW level.
- Always verify timer API compatibility with the installed Arduino-ESP32 version.

23. References

Course source

Week 03 slides: Event-Driven Precision — Interrupts, Hardware Timers, and Switch Debouncing on the ESP32. Used as the conceptual basis for the Week 4 progression, especially the slides on mechanical bounce, debouncing architectures, timer mathematics, timer initialization and non-blocking timer debounce.

Official technical references

- **Arduino-ESP32 Timer API:** https://docs.espressif.com/projects/arduino-esp32/en/latest/api/timer.html

- **Arduino-ESP32 GPIO API:** https://docs.espressif.com/projects/arduino-esp32/en/latest/api/gpio.html
- **Arduino-ESP32 2.x → 3.0 Migration Guide:** https://docs.espressif.com/projects/arduino-esp32/en/latest/migration_guides/2.x_to_3.0.html

VERSION NOTE

The examples in this guide target the current Arduino-ESP32 timer API documented by Espressif. If PlatformIO resolves a different Arduino-ESP32 version, verify the API in the build output and documentation before debugging the code.